

Trabajo Práctico 1: Inteligencia Artificial y Neurociencias

Aprendizaje por Refuerzos

Rosa Tagle, Juana Arslanian, Luciana Alarcón y Varvara Mironov

2 de septiembre de 2026

1 Objetivo

El objetivo fue entrenar un agente para jugar **Gold Dice RL** y maximizar el puntaje final de una partida de 30 turnos. En cada turno los dados producen oro y el agente decide si convertirlo en puntos, conservarlo, invertirlo para aumentar la producción futura o comprar protección contra las tormentas. Una inversión da recompensa inmediata cero, pero puede permitir obtener más puntos varios turnos después.

Comparamos **Q-Learning**, **SARSA**, **Monte Carlo** y **DQN** con dos referencias: **Random Legal**, que elige al azar entre las acciones válidas, y **Simple Expectancy**, una heurística que compra escudo si no tiene, invierte si quedan suficientes turnos y puntua todo en el último turno. Nuestra hipótesis fue que las políticas aprendidas superarían a Random Legal al aprovechar la experiencia de entrenamiento. También evaluamos si podían superar a Simple Expectancy, que ya incorpora conocimiento del juego. Comparamos todos los modelos y elegimos el que obtuvo el mayor puntaje promedio para el torneo.

2 Estado, acciones y recompensa

El estado es la información con la que el agente distingue situaciones y consulta sus valores aprendidos. Si conserva demasiado detalle, una tabla acumula muchas combinaciones visitadas pocas veces, mientras que si agrupa demasiado, mezcla situaciones que pueden requerir acciones distintas. Por eso no existe necesariamente una única codificación óptima pues cada agente debe conservar la información útil sin fragmentar excesivamente la experiencia.

Representamos el estado según el modelo:

- **Q-Learning y SARSA:** Turnos restantes, oro logarítmico, cuatro indicadores de compras posibles, dados y bonus. Unificarlos permite comparar ambos métodos con igual información y evita el gran número de estados producido por usar oro y turno exactos.
- **Monte Carlo:** Turnos restantes, oro agrupado mediante 16 umbrales, dados y bonus. Su estado compacto pierde detalles, como el máximo de la tirada, pero acumula más visitas por estado y estima mejor el retorno completo.
- **DQN:** Ocho valores normalizados: turno, oro, dados, bonus, escudos, valor guardado, suma y máximo de la tirada. La red conserva valores numéricos y puede generalizar entre observaciones cercanas sin discretizarlas en una tabla.

Cuadro 1: Representación del estado por modelo más cercana al código

Modelo	Estado
Q-Learning SARSA	y (turns_left, gold_level, (can_store, can_shield, can_upgrade, can_buy_die), num_dice, dice_bonus, shields, roll_max) Los dados y el bonus se limitan a 10, los escudos a 2 y el máximo de la tirada a 12.
Monte Carlo	(turns_left, num_dice, dice_bonus, shields, gold_bucket) Los dados y el bonus se limitan a 8, los escudos a 2 y el oro se agrupa mediante 16 umbrales.
DQN	[turn, gold, num_dice, dice_bonus, shields, stored_value, roll_sum, roll_max] Los ocho valores se normalizan antes de ingresar a la red neuronal.

Fuimos probando distintas opciones y en versiones anteriores cada agente utilizaba una representación diferente. **SARSA** conservaba valores casi exactos y generaba una tabla muy grande. **Q-Learning** ya agrupaba

algunas variables. **Monte Carlo** usaba un estado más compacto y **DQN** recibía valores numéricos normalizados. Intentamos unificar la representación de los métodos tabulares, pero el cambio no benefició a todos por igual. Q-Learning mejoró al incorporar información relacionada con las compras, mientras que SARSA disminuyó ligeramente su desempeño.

Monte Carlo se entrenó con 100.000 episodios en ambas versiones. Con su estado compacto guardó 17.540 estados diferentes, mientras que con la representación unificada superó los 63.000. Al repartir la misma experiencia entre más estados, cada uno fue visitado menos veces y las estimaciones del retorno empeoraron. Por eso mantuvimos el estado compartido sólo para Q-Learning y SARSA. DQN conservó su vector numérico porque la red neuronal puede generalizar entre valores cercanos.

No incluimos los puntos en los estados tabulares porque no modifican las transiciones futuras, ni la suma de la tirada porque ya fue incorporada en el oro.

Las acciones son PASS, BUY_DICE, UPGRADE, BUY_SHIELD, STORE_BEST_DIE y SCORE_ALL, donde esta última convierte todo el oro en puntos y evita tratar cada monto parcial como una acción diferente. En cada estado, las que no pueden pagarse o ejecutarse se excluyen.

La recompensa es exactamente el oro convertido en puntos. Comprar o esperar da recompensa inmediata cero, aunque puede mejorar el retorno futuro. No agregamos premios artificiales, por lo que maximizar el retorno coincide con maximizar el puntaje del juego.

3 Métodos y entrenamiento

Q-Learning guarda un valor por cada par estado-acción y actualiza

$$Q(s, a) \leftarrow Q(s, a) + \alpha \left[r + \gamma \max_{a'} Q(s', a') - Q(s, a) \right].$$

Es *off-policy* porque aprende usando la mejor acción estimada del estado siguiente.

SARSA usa la próxima acción que realmente elegirá su política, por ende es *on-policy* y considera el efecto de su propia exploración.

Monte Carlo espera al final del episodio y actualiza las decisiones según el retorno completo observado.

DQN reemplaza la tabla por una red con dos capas ocultas de 64 unidades. Estima un valor por acción y ajusta sus pesos para reducir el error entre esa predicción y el objetivo $r + \gamma \max_{a'} Q(s', a')$. Entrena con lotes aleatorios de una memoria de experiencias y una red objetivo actualizada periódicamente, lo que reduce la inestabilidad.

4 Hiperparámetros

- **Gamma:** Es el factor de descuento que controla cuánto importan las recompensas futuras. Cuanto más cerca de 0 está, más importan las recompensas inmediatas.
- **Alpha:** Es la tasa de aprendizaje que controla cuánto cambia Q con cada experiencia. Con un alpha pequeño, aprende lentamente y presenta valores estables, mientras que con un alpha grande aprende rápidamente, pero puede ser inestable.

Cuadro 2: Comparativa de modelos y sus configuraciones

Modelo	Episodios	Aprendizaje	γ	Configuración adicional
Q-Learning	1.000.000	$\alpha = 0,05$	1,00	ε : 1,00 a 0,05
SARSA	100.000	$\alpha = 0,10$	1,00	Selección entre 12 combinaciones
Monte Carlo	100.000	$\alpha = 0,05$	1,00	Decaimiento de ε en el 80 % inicial
DQN	100.000	$lr = 0,001$	1,00	Buffer 50.000, lote 64, red objetivo cada 500 actualizaciones

De este modo, usamos $\gamma = 1.0$ porque la partida es finita y todos los puntos obtenidos dentro de los 30 turnos importan por igual. En Monte Carlo este valor está implícito al sumar todas las recompensas futuras sin descuento.

En los métodos tabulares, α controla cuánto cambia cada valor con una nueva experiencia. Para Q-Learning y Monte Carlo usamos $\alpha = 0,05$, que incorpora aproximadamente un 5 % del error observado en cada actualización, y para SARSA probamos $\alpha \in \{0,10; 0,05; 0,01\}$ y $\gamma \in \{1; 0,99; 0,95; 0,90\}$, donde la validación seleccionó $\alpha = 0,10$ y $\gamma = 1$.

DQN no actualiza una celda de una tabla, sino los pesos de una red neuronal. Por eso utiliza un *learning rate* de 0,001, que cumple un papel similar a α .

Durante el entrenamiento se usó una política ε -greedy que, con probabilidad ε el agente explora una acción válida y, en caso contrario, elige la mejor acción estimada. En la evaluación se utilizó $\varepsilon = 0$, por lo que los agentes actuaron sin exploración.

5 Resultados

Evaluamos todos los agentes en las mismas 1.000 partidas, con semillas 0 a 999. Cada valor es el puntaje final por partida.

Cuadro 3: Rendimiento en 1.000 partidas.

Agente	Media	Desvío	Mediana	Mínimo	Máximo
DQN	596,50	123,10	603,50	114	1.003
Simple Expectancy	347,45	102,33	345,00	54	666
Monte Carlo	322,48	63,57	322,00	96	596
Q-Learning	208,03	22,07	210,00	108	259
SARSA	106,32	9,80	106,00	75	139
Random Legal	63,47	38,20	56,00	0	317

La media representa el desempeño habitual, mientras que el máximo puede corresponder a una partida excepcional. DQN fue el único método aprendido que superó a Simple Expectancy. Monte Carlo y Q-Learning también superaron ampliamente a Random Legal y aunque SARSA lo superó, aprendió una política estable y demasiado conservadora.

6 Análisis y conclusión

Nuestra hipótesis se cumplió parcialmente. Todos los agentes aprendidos superaron a **Random Legal**, pero solamente **DQN** superó a **Simple Expectancy**. Esto muestra que utilizar experiencia no garantiza por sí solo superar una estrategia sencilla pero bien diseñada. No obstante, DQN sí logró aprovechar la experiencia y la generalización para superarla.

También son fundamentales la representación del estado y la capacidad del modelo para generalizar. Simple Expectancy resultó ser un baseline exigente porque incorpora directamente una estrategia razonable para el juego y expuso que los métodos tabulares resultaron inferiores a esa heurística. Aunque la discretización mantiene manejable la tabla, agrupa situaciones que pueden requerir decisiones distintas. Por ejemplo, **Q-Learning** se benefició de incluir indicadores relacionados con las compras, y **Monte Carlo** funcionó mejor con un estado compacto ya que al intentar una representación más detallada, la experiencia quedó repartida entre demasiados estados y sus estimaciones empeoraron. **DQN** conservó más información y, a la vez, generalizó entre situaciones parecidas mediante la red neuronal.

Además, para los métodos tabulares fue difícil asignarle correctamente a una compra con recompensa inmediata cero el mérito por puntos obtenidos muchos turnos después.

Concluimos que el desempeño depende tanto del algoritmo como de la información del estado y su capacidad de generalización. Asimismo, vimos que un algoritmo de aprendizaje por refuerzos no supera automáticamente a una buena heurística. La evaluación común y reproducible permitió seleccionar **DQN** como agente final por su promedio de 596,50 puntos.